

Lab 3: Architectural Pattern Justification

Problem Statement #40 – Student Startup Crowdfunding Platform

Architecture Selection: We chose **Microservices Architecture** for the **Student Startup Crowdfunding Platform**.

Architectural Choice

The platform is decomposed into independently deployable components: Web/Mobile UI, Campaign Manager (UC-01, 02, 04), Payment Service (UC-03, 05, 06, 10), Milestone Verification (UC-07, 08, 09), Notification, and a Database, plus the external Payment Gateway. They interact only through published interfaces (REST/HTTPS, an asynchronous message queue, SQL/JDBC), which matches the provided/required interface notation in the component diagram. Layered and Client-Server were rejected because both keep payment logic in one deployable unit that scales as a whole and fails as a whole.

Reason 1 – Isolating the milestone-gated financial workflow (FR-001, FR-004)

The core rule of the platform is that funds move only after faculty sign-off. In our design the Payment Service exposes IFundRelease, and it is invoked solely by Milestone Verification after the Faculty Committee records “Approve” (UC-09 including UC-10). Because fund movement is a separate service with one narrow entry point, no other module (for example, Campaign Manager) can disburse escrow, and every release, rejection, or refund can be written to the immutable audit log in one place.

Reason 2 – Uneven load and independent change (FR-005, NFR-002)

Pledge and campaign-page traffic spikes when a campaign launches, whereas faculty review is low-volume and notifications are event-driven. Microservices let us scale Campaign Manager and Payment Service for the peak without scaling Milestone Verification, and let each service change (for example, adding new EarlyBird discount rules in UC-06) without redeploying the platform.

Security Advantage (NFR-001)

Only the Payment Service holds payment tokens and communicates with the external PCI-DSS Payment Gateway (IGateway over HTTPS). Raw card numbers are never stored or passed to any other component, and the Database stores tokens and transaction references only. This confines PCI-DSS scope to a single service, so a compromise of the UI, Campaign Manager, or Notification service cannot expose cardholder data. Inter-service calls use authenticated HTTPS, and Faculty-only access is enforced at Milestone Verification, so non-faculty accounts cannot trigger tranche releases (FR-004 fail condition).

Performance Benefit (NFR-002, FR-005)

Each service scales horizontally on its own, so the target of 500 concurrent page views and pledges with p95 response time under 1.5 s is met by adding only Campaign Manager and Payment Service instances. Notifications go through an asynchronous queue, so email or in-app delivery (within 30 seconds) never blocks the pledge or approval flow.

Trade-offs Accepted

Microservices add network latency, operational complexity, and cross-service consistency risk. We mitigate these with idempotent payment and release calls (a retried releaseTranche() cannot pay twice), a shared audit log, and asynchronous retries for notifications. Given that the system handles other people’s money and needs verifiable accountability, these costs are acceptable.